



正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台 : www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://openedv.taobao.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。



扫码关注正点原子公众号



扫码下载“原子哥”APP

目录

SPL 的设备树镜像	3
1.1 概述	3
1.2 如何生成 MiniloaderAll.bin 镜像	4

SPL 的设备树镜像

1.1 概述

通过 RK 提供的 make.sh 脚本编译 u-boot 之后, 会生成 u-boot 镜像、同样也会生成 SPL 的镜像, 会在 u-boot 目录下生成一个名为 spl 的目录, 如下所示:

```
alientek@android:~/hdd/rk3568-a/u-boot/spl$ ls
arch  cmd  disk  dts  fs  lib  u-boot-spl  u-boot-spl.dtb  u-boot-spl.lds  u-boot-spl-nodtb.bin
board common drivers env include u-boot.cfg u-boot-spl.bin u-boot-spl-dtb.bin u-boot-spl.map u-boot-spl.sym
alientek@android:~/hdd/rk3568-a/u-boot/spl$
```

因为设备树采用的是独立方式, 所以设备树镜像并不会直接嵌入到 SPL 镜像中, 而是会生成一个单独的设备树镜像文件。

在上图中可以看到有多种 .bin 和 .dtb 文件:

u-boot-spl.dtb 就是 SPL 的设备树镜像文件, 单独编译出来。

u-boot-spl.lds 这个是编译 SPL 使用的链接脚本, 需要注意, u-boot 目录下的 spl 文件夹是在编译过程中创建出来的, 所以 u-boot-spl.lds 文件是从其它地方拷贝过来的。

u-boot-spl-nodtb.bin 这个是 SPL 镜像 不包含设备树镜像, 因为采用独立方式编译设备树镜像, 所以设备树镜像不会嵌入到 u-boot 的镜像中。

而 u-boot-spl.bin 和 u-boot-spl-dtb.bin 其实是同一个文件, Copy 而来, 这两个文件其实包含了设备树镜像 u-boot-spl.dtb 的 u-boot 镜像文件, 但是主要这个设备树镜像他不是嵌入进去 (如果是嵌入方式, 那么会将设备树的镜像单独放在 u-boot 镜像的某一个段中), 而是会将其附加在 u-boot 镜像后面, 所以设备树镜像就直接在 u-boot 镜像后面。

所以由此可知, u-boot 中编译 SPL 之后, 是会生成三种镜像: SPL 的设备树镜像 u-boot-spl.dtb、SPL 镜像 (不包含设备树镜像) u-boot-spl-nodtb.bin、SPL 镜像 (包含 u-boot 镜像) u-boot-spl-dtb.bin。

而 u-boot-spl.bin 是由 u-boot-spl-dtb.bin 拷贝出来的。而后续制作 Miniloader.bin 镜像时, 会使用 u-boot-spl.bin 镜像文件, 也就是将 u-boot-spl.bin 作为 SPL, 再加上 TPL, 最终生成 Miniloader.bin 镜像文件。

嵌入方式和独立方式的不同会在 fdtdec_setup 函数中体现出来:

```

int fdtdec_setup(void)
{
    #if CONFIG_IS_ENABLED(OF_CONTROL)
    # if CONFIG_IS_ENABLED(MULTI_DTB_FIT)    //没定义
        void *fdt_blob;
    # endif
    # ifdef CONFIG_OF_EMBED    //没定义 表示dtb是嵌入方式
        /* Get a pointer to the FDT */
    # if CONFIG_SPL_BUILD
        gd->fdt_blob = __dtb_dt_spl_begin; //如果是嵌入方式 会在一个特定的段
    # else
        gd->fdt_blob = __dtb_dt_begin;
    # endif
    # elif defined CONFIG_OF_SEPARATE //我们是这种方式 表示独立
    # if CONFIG_SPL_BUILD
        /* FDT is at end of BSS unless it is in a different memory region */
        if (IS_ENABLED(CONFIG_SPL_SEPARATE_BSS))
            gd->fdt_blob = (ulong *)&_image_binary_end; //设备树镜像位于SPL镜像的后面
        else
            gd->fdt_blob = (ulong *)&__bss_end;
    # else
        /* FDT is at end of image */
        gd->fdt_blob = (ulong *)&_end;

    # if CONFIG_USING_KERNEL_DTB
        gd->fdt_blob_kern = (ulong *)((ulong)gd->fdt_blob +
            ALIGN(fdt_totalsize(gd->fdt_blob), 8));
    # endif
    # endif
}

```

SPL 的设备树是比较小的, 它也是使用了 u-boot 本身的设备树文件, 但是在编译的时候会进行处理, 将包含了 "u-boot,dm-spl" 或 "u-boot,dm-pre-reloc" 属性的设备节点给过滤出来, 其余的节点会被剔除, 这样做的原因在于减小整个 MiniloaderAll.bin 镜像的大小。

1.2 如何生成 MiniloaderAll.bin 镜像

生成 MiniloaderAll.bin 文件非常简单, 由 RK 提供的工具制作而成, 这个在 RK 提供的文档上有说明: 使用 boot_merger 工具, 该工具在 rkbin/tools 目录下, 该工具的使用方法很简单,

./tools/boot_merger [ini file]

执行 boot_merger 命令时, 指定一个 ini 文件即可! 在这个 ini 文件中会指定 SPL 镜像、TPL 镜像等, 在 rkbin/RKBOOT 目录下有很多 ini 文件, 譬如以 RK3568 为例 (rkbin/RKBOOT/RK3568MINIALL.ini):

```

[CHIP_NAME]
NAME=RK3568 -----芯片名称: "RK"加上与maskrom约定的芯片型号
[VERSION]
MAJOR=1 -----主版本号
MINOR=1 -----次版本号
[CODE471_OPTION] -----code471, 目前设置为ddr.bin
NUM=1
Path1=bin/rk35/rk3568_ddr_1560MHz_v1.09.bin
Sleep=1
[CODE472_OPTION] -----code472, 目前设置为usbplug.bin
NUM=1
Path1=bin/rk35/rk356x_usbplug_v1.10.bin
[LOADER_OPTION]
NUM=2
LOADER1=FlashData -----flash.data, 目前设置为ddr.bin
LOADER2=FlashBoot -----flash.boot, 目前设置为miniloader.bin
FlashData=bin/rk35/rk3568_ddr_1560MHz_v1.09.bin
FlashBoot=bin/rk35/rk356x_spl_v1.11.bin
[OUTPUT] -----输出文件名
PATH=rk356x_spl_loader_v1.09.111.bin
[SYSTEM]
NEWIDB=true
[FLAG]
471_RC4_OFF=true
RC4_OFF=true

```

ddr bin 就是 TPL、miniloader bin 就是 SPL, 事实上除了 TPL 和 SPL 之外, 还有一个 usbplug bin, 这个为了让 maskrom 支持 usb 功能而添加的一个闭源的二进制镜像文件。

以上这个图是 RK3568 使用了闭源的 TPL+SPL, 这些镜像都在 rkbin/bin/rk35 目录下, 如下所示:

```

alientek@android:~/hdd/rk3568-a/rkbin/bin/rk35$ ls
FlashHead.bin          rk3568_ddr_630MHz_v1.09.bin  rk3568_b131_ultra_v2.06.elf  rk3568_ddr_528MHz_v1.09.bin  rk356x_spl_nand_v1.07.bin
rk3568_ddr_1056MHz_ultra_print_off_v1.07.bin  rk3568_ddr_780MHz_ultra_v1.07.bin  rk3568_b132_v1.05.bin  rk3568_ddr_630MHz_v1.09.bin  rk356x_spl_v1.11.bin
rk3568_ddr_1056MHz_v1.09.bin  rk3568_ddr_780MHz_v1.09.bin  rk3568_ddr_1056MHz_v1.09.bin  rk3568_ddr_780MHz_v1.09.bin  rk356x_usbplug_nand_v1.04.bin
rk3568_ddr_528MHz_ultra_print_off_v1.07.bin  rk3568_ddr_920MHz_v1.09.bin  rk3568_ddr_1184MHz_v1.09.bin  rk3568_ddr_920MHz_v1.09.bin  rk356x_usbplug_v1.10.bin
rk3568_ddr_528MHz_ultra_v1.07.bin  rk3568_ddr_920MHz_ultra_v1.07.bin  rk3568_ddr_1332MHz_v1.09.bin  rk3568_ramboot_null10.bin  UsbHead.bin
rk3568_ddr_528MHz_v1.09.bin  rk3568_ddr_920MHz_v1.09.bin  rk3568_ddr_1560MHz_v1.09.bin  rk3568_ramboot_v1.08.bin
alientek@android:~/hdd/rk3568-a/rkbin/bin/rk35$

```

可以看到这个目录下有很多的 rk3568_ddr_xxx 的镜像文件, 这些其实就是 TPL 镜像 (用于对 DDR 进行初始化操作, ddr bin); 这些 rk356x_spl_x 的文件便是 SPL 镜像文件; 这就是 RK 平台所提供的闭源的镜像, 只有二进制镜像文件、没有其源码。

如果我们要使用开源的 SPL, 则需要将 FlashBoot 指定为我们通过 U-Boot 编译出来的 u-boot-spl.bin 镜像文件, 事实上, u-boot 源码中 RK 提供的 make.sh 脚本帮我们做好了, 我们只需执行一个命令即可,

`./make.sh --spl` 表示使用 (U-Boot 开源 SPL+闭源 TPL) 制作 MiniloaderAll.bin 文件, 执行这个命令之后会直接将 FlashBoot 指定为 u-boot 目录下 spl/u-boot-spl.bin 文件。

当然也可以这样:

`./make.sh --spl --tpl`

这样表示同时使用开源 TPL+开源 SPL 制作 MiniloaderAll.bin, 当然, 对于 RK3568 平台, u-boot 编译出来的 TPL 并不可使用, 貌似是 RK 没做支持。

执行命令成功之后, 便会在 u-boot 目录下生成对应的 MiniloaderAll.bin 镜像文件, 不过名字并不是 MiniloaderAll.bin, 而是 rk356x_spl_loader_v1.09.111.bin, 这个直接重命名即可!

```

include  MAINTAINERS  PREUPLOAD.cfg
Kbuild   Makefile      README
Kconfig  make.sh       rk356x_spl_loader_v1.09.111.bin
lib       net         scripts
Licenses post        snapshot.commit

```

事实上, 但我们执行

```
./make.sh rk3568
```

编译 u-boot 源码的时候, 编译完 u-boot 源码之后, 该脚本会自动帮我们生成 MiniloaderAll.bin, 当然名字也是 rk356x_spl_loader_v1.09.111.bin, 但是要注意, 此时它是使用闭源 TPL+闭源 SPL 来生成的, 也就是 rkbin/RKBOOT/RK3568MINIALL.ini 文件。